Database Replication

Database Replication is the process of copying and maintaining database data across multiple servers to improve availability, performance, and disaster recovery. The two primary patterns are **master-slave** (also called primary-replica) and **master-master** (multi-master) replication.

MASTER-SLAVE REPLICATION (PRIMARY-REPLICA)

How it works: One server is the **master/primary** (accepts writes), others are **slaves/replicas** (read-only copies that receive updates from master).

Data flow:

1. Application writes to master
2. Master logs the change
3. Master sends change to all slaves
4. Slaves apply the change to their data
5. Application reads from any slave (or master)

Types of Master-Slave Replication

1. **Synchronous Replication :** Master waits for slaves to acknowledge they've received and applied the change before confirming the transaction.

```
Timeline:
T1: Application writes to master
T2: Master sends to all slaves
T3: Slaves acknowledge receipt
T4: Slaves apply change
T5: Slaves send "done" acknowledgment
T6: Master confirms transaction to application ← Application waits here

Total latency: Network roundtrip + slave processing time
```

```
-- postgresql.conf on master
synchronous_standby_names = 'slave1,slave2'   -- Wait for these slaves
synchronous_commit = on

-- Every commit waits for slaves to acknowledge
```

Advantages:

- **Strong consistency**: Slaves always have latest data
- **No data loss**: If master fails, slaves are up-to-date
- **Read-your-writes**: User sees their changes immediately on any server

Disadvantages:

- **High latency**: Every write waits for all slaves
- **Availability** impact: If slave is slow/down, writes slow/fail
- **Not scalable**: Adding slaves increases write latency

Best For:

- Financial Systems
- Small number of slaves
- Write latency is acceptable
- Strong consistency is critical

2. **Asynchrounous Replication :** Master doesn't wait for slaves. Changes are sent to slaves, but master continues immediately.

```
Timeline:
T1: Application writes to master
T2: Master confirms transaction to application ← Fast response!
T3: Master sends to slaves (in background)
T4: Slaves eventually apply change

Application doesn't wait for replication
```

Advantages:

- **Low latency:** Writes complete quickly
- **High availability:** Slave failures don't impact writes
- **Scalability:** Add many slaves without affecting write performance

Disadvantages:

- **Replication lag:** Slaves may be seconds/minutes behind
- **Data loss risk:** If master crashes, un-replicated changes lost
- **Eventual consistency:** Reads might return stale data

Best For:

- Most Web Applications
- Many slaves needed
- Write performance critical
- Brief data staleness acceptable

3. **Semi-Synchronous Replication :** Master waits for **at least one** slave to acknowledge, then continues. Best of both worlds.

```
Timeline:
T1: Application writes to master
T2: Master sends to all slaves
T3: First slave acknowledges ← Master waits only for this
T4: Master confirms transaction to application
T5: Other slaves apply change (eventually)

Wait for one slave, not all
```

Advantages:

- **Better durability:** At least one copy confirmed before commit
- **Lower latency:** Faster than full synchronous
- **Reasonable availability:** One slave failure doesn't block writes

Disadvantages:

- **Slightly higher latency:** Compared to async
- **Partial guarantees:** Not as strong as full sync
- **Configuration complexity:** Choose which slaves to wait for

Best For:

- Production systems balancing consistency and performance
- Need some durability guarantee without full sync cost
- 2 to 3 slaves in same region

Replication Methods

1. **Statement-based replication** : Master logs SQL statements, slaves replay them.
2. **Row-based replication :** Master logs the actual data changes (before/after row values), slaves apply those exact changes.
3. **Mixed/Hybrid Replication :** Database automatically chooses statement or row-based per query.

Use Cases of Master-slave

1. Read scaling
2. Geographic distribution
3. Analytics/Reporting don’t slow production
4. Backup

What happens when master fails

1. **Manual failover**

```
1. Master crashes/fails
2. Admin detects failure
3. Admin promotes slave to master
4. Update application config to point to new master
5. Other slaves replicate from new master

Downtime: Minutes to hours (manual intervention)
```

2. **Automatic failover**

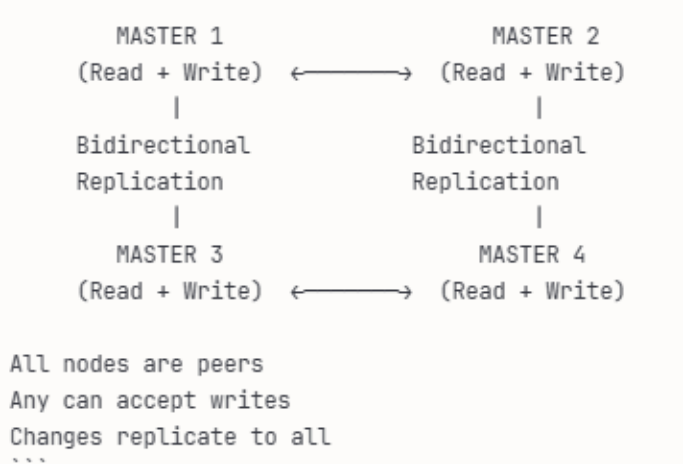
```
1. Master crashes/fails
2. Monitoring detects failure (heartbeat timeout)
3. Automation promotes most up-to-date slave
4. DNS/load balancer updated automatically
5. Other slaves switched to new master

Downtime: Seconds to minutes (automated)
```

Master-Slave Limitations

1. Write bottleneck
2. Single point-of-failure
3. Replication lag

MASTER-MASTER REPLICATION (MULTI-MASTER)

How it works: Multiple servers can accept writes, and they replicate changes to each other bidirectionally.



Data flow:

```
User writes to Master 1:
  → Master 1 replicates to Masters 2, 3, 4

User writes to Master 2:
  → Master 2 replicates to Masters 1, 3, 4

All masters eventually consistent
```

Types of Multi-Master:

1. **Active-Active (All masters Active)**

Benefits:

- Write scalability: Distribute writes across masters
- High availability: Any master can fail
- Low latency: Write to nearest master
- Geographic distribution: Masters in multiple regions

Challenges:

- **Conflict resolution:** Two users update same row on different masters
- **Complexity:** Much harder to manage than master-slave
- **Consistency:** Eventual consistency only

2. **Active-Passive (One Master Active, Others Standby)**

```
**Setup:**
```
Primary Master (active) ⟷ Secondary Master (passive)
         | |
   All writes here   Stays in sync
                                 (for failover)
```
```

Benefits:

- **Fast failover :** Simply activate secondary
- **Consistency :** Only one accepts writes (no conflicts)
- **Simple :** Easier than active-active

Limitations:

- Does not solve write scaling

Replication vs Sharding (Quick Compare)

Feature	Replication	Sharding
Purpose	Availability & reads	Horizontal scaling
Data	Same data everywhere	Different data per node
Writes	Usually single primary	Distributed
Complexity	Lower	Higher